Verifiable Work Product

An Open Package Format for Evidence-Backed Professional Conclusions

Pat White (Bitwave) and the VWP contributors

Version 0.1 draft — July 2026 — request for comment

> *VWP validation is not an audit opinion, legal opinion, accounting opinion, or assurance engagement. It means only that a package conforms to specified schemas and validation rules. Professional judgment and applicable standards still govern reliance on any conclusion. Nothing in this paper is accounting, audit, or legal advice.*

Abstract

Professional conclusions — revenue memos, reconciliations, control tests, valuation support, workpapers of every kind — are produced as documents but consumed as trust. The document formats we use today (PDFs, spreadsheets, records inside proprietary applications) carry the narrative of a conclusion while discarding its support structure: the evidence, the extracted facts, the calculations, the judgment calls, and the checks that were or were not performed. Anyone downstream must either trust the author, trust the tool, or redo the work.

AI has turned this from an inefficiency into a structural risk. Models now draft plausible professional work products at near-zero marginal cost, and plausibility is precisely the problem: fluent output is not verified output. The binding constraint in professional workflows is shifting from drafting to review, and document formats give reviewers nothing to verify mechanically.

VWP (Verifiable Work Product) is an open package format and verification framework that responds by changing the artifact rather than the reviewer. A `.vwp` file is a ZIP package that bundles evidence references, extracted facts, claims, judgments, calculations, validation results, approvals, provenance, and human-readable renderings into a single portable, machine-verifiable unit. The rendered memo is a derived view; the structured package is the source of truth. Deterministic validators — never the model that drafted the package — check integrity, schema conformance, evidence linkage, mathematical tie-outs, and domain completeness rules, and they surface the professional judgments that machines must not decide. Where review is analytical rather than mechanical, VWP supports *analysis attestations*: structured, signed records of model-assisted review procedures, kept strictly distinct from deterministic validation.

This paper motivates the format, describes its object model and verification model, walks through a worked example, situates VWP in audit workflows, and states its limitations plainly. A reference implementation — schemas, a verifier CLI, and synthetic example packages — is available in the open VWP repository.

1. The problem: work products you cannot verify

1.1 Conclusions detached from their support

Consider a routine professional artifact: a revenue recognition memo, a period-end reconciliation, a test of an internal control, a fair value support schedule. Each one is, in substance, a *conclusion resting on a structure*: source evidence, facts extracted from that evidence, calculations over those facts, points where professional judgment was required, and checks that the pieces hang together.

In practice, that structure is destroyed at the moment of delivery. The conclusion ships as a PDF or a spreadsheet; the evidence lives in email threads, shared drives, and screenshots; the calculations live in cells whose provenance is invisible; the judgments live in the author's head or in a sentence of narrative; and the checks — if they ran at all — leave no artifact. The workpaper documents *what was concluded* and gestures at *why*, but nothing downstream can mechanically confirm that the pieces referenced actually exist, actually say what the memo claims they say, or actually add up.

This leaves every downstream consumer — a reviewer, an auditor, a regulator, a successor preparer, an acquirer's diligence team — with three options:

1. **Trust the author.** Reasonable when volume is low and authors are known; increasingly untenable otherwise.
2. **Trust the tool.** The conclusion is only as verifiable as the vendor's application is transparent, and it usually is not. The artifact cannot be checked outside the system that produced it.
3. **Redo the work.** Reliable and ruinously expensive — which is why it is done on samples, and why so much professional review is in fact option 1 wearing the clothes of option 3.

1.2 AI breaks the review model

The traditional review model carries two hidden assumptions: drafting is expensive, and drafts are mostly right, because a competent professional produced each one with care roughly proportional to its difficulty.

Large language models invalidate both assumptions at once. Drafting cost collapses toward zero, so volume rises to fill whatever capacity exists. And the failure mode changes character: model-drafted work is not sloppy in recognizable ways but *fluently, confidently, occasionally wrong* — wrong in the citation that doesn't exist, the number that doesn't tie, the evidence that was never actually examined. The signals experienced reviewers use to allocate skepticism (rough drafting, hedged language, visible effort) are exactly the signals models erase.

Professions have two reflexive responses available, and both fail. Banning AI drafting forfeits real leverage and, in practice, merely drives it underground. Trusting AI drafting — "the model is usually right" — imports an unbounded, unauditable error rate into work whose entire value is reliability.

There is a third response: **change the artifact, not the reviewer**. If a work product ships as a structured package whose mechanical properties — evidence exists and matches its hash, references

resolve, calculations tie, required parts are present, judgments are flagged — can be verified by deterministic software, then it no longer matters *who or what drafted it*. The generator becomes untrusted by design. Machines verify structure exhaustively; humans concentrate on the judgments, which the package is required to surface rather than bury.

This is the same architectural move the software industry made for its own supply chain: signed, attestation-carrying artifacts verified independently of the systems that built them. Professional work products need their equivalent.

1.3 The economics of the review bottleneck

The cost structure of evidence-backed work is dominated not by analysis but by *assembly and re-verification*. Preparers spend large fractions of close and audit cycles locating evidence, reconciling versions, and responding to document requests; auditors spend correspondingly large fractions requesting, re-requesting, and manually inspecting what arrives. The same support is assembled, checked, lost, and reassembled at every handoff — preparer to reviewer, client to auditor, year to year — because the artifact that crosses each boundary carries no machine-checkable structure.

Sampling itself is largely an artifact of this cost structure. Auditors examine subsets not because subsets are epistemically ideal but because manual verification does not scale. Where checks become machine-executable, the economics invert: structural verification — do the references resolve, do the hashes match, do the schedules tie, is every required element present — can run on *every* package at negligible marginal cost, on receipt and again at any later date. Human attention, the genuinely scarce resource, is then spent where it is irreplaceable: evaluating sufficiency, appropriateness, and judgment.

AI raises the stakes on both sides of this ledger. It floods the review channel with cheap drafts, and it makes cheap *verification* possible — but only if work products are structured so that verification is a computation rather than a reading.

---

2. The thesis

> *A professional conclusion should be represented as a portable, signed, evidence-linked, machine-verifiable package containing evidence, extracted facts, claims, judgments, calculations, validations, approvals, provenance, and human-readable renderings.*

A conforming package should let any independent party answer, without access to the system that produced it:

1. What conclusion was reached?
2. What evidence supports it?
3. What facts were extracted from that evidence?
4. What claims were made, and what supports each claim?
5. What judgments were required, and on what basis?

6. What calculations were performed, and are they recomputable?
7. What validations passed, failed, or require review?
8. Who reviewed or approved it, and in what capacity?
9. What tools, models, and policies produced it?
10. Does the human-readable rendering correspond to the structured content?

Five design principles follow:

- **Structured over rendered.** The memo a human reads is a derived view. The structured package is the source of truth, and verifiers verify structure, not prose.
- **Deterministic verification.** Core verification is schema validation, hash checking, reference-graph resolution, and arithmetic — no network access, no model inference, no execution of package contents.
- **Untrusted generators.** AI may extract facts, draft claims, and render explanations. Nothing in verification depends on the generator being honest, competent, or even identified.
- **Model-agnostic.** Provenance records which tools and models produced a package; verification never requires them. The format must outlive any vendor or model generation.
- **Domain-extensible.** The core is generic. Profiles add domain vocabulary (financial assertions); extensions add narrower validators (ASC 606 tie-outs, wallet control checks, control-test completeness, forecast assumption and freshness checks).

Taken together, these principles yield something more than a file format: **VWP creates a structured evidence object that multiple independent agents — deterministic validators, preparer LLMs, auditor LLMs, and human reviewers — can analyze, challenge, sign, and append to over time.** The package is not a snapshot of one author's conclusion but a substrate that accumulates verification: each validator run, analysis attestation, approval, and revision attaches to the same evidence-linked structure, and each is independently checkable by the next agent to arrive.

3. The VWP model

3.1 The object chain

Every VWP package embodies one information flow:

```
evidence -> facts -> claims -> judgments / calculations -> validations -> conclusion -> render
```

- **Evidence** is source material: documents, exports, statements, screenshots — either included in the package or referenced externally with a locator, each identified by SHA-256 content hash where possible.
- **Facts** are extracted observations grounded in evidence: "the order form states a total contract value of 300,000 USD." Facts are observations, not conclusions; each carries its evidence references, extraction method, and confidence.

- **Claims** are the assertions that constitute the work product: each links to supporting facts, evidence, calculations, and judgments, and may carry references to professional standards and assertion vocabulary. Claims form a graph a verifier can walk.
- **Judgments** are first-class objects for the points where professional discretion was required: the question, the proposed conclusion, its basis, the alternatives considered, a risk level, and the role required to approve it. VWP's position is that judgment cannot be validated away — but it can be *surfaced, structured, and made impossible to overlook*.
- **Calculations** declare their inputs, outputs, and method, and are ideally recomputable by a verifier that never saw the system that produced them.
- **Validations** record which deterministic checks ran and what they found.
- **Approvals** record human review actions — approved, rejected, changes requested, acknowledged — as review state, never as audit signoff.
- **Provenance** records who and what produced the package: people, agents, toolchain, models, policy versions, and source systems.
- **Renderings** are the human-readable views — the memo, the workpaper — derived from the structured content.

3.2 The package

A `.vwp` file is a ZIP archive with a declared layout:

```
example.vwp
  manifest.json                 required — identity, profile, parts, SHA-256 hashes
  workproduct.json              required — what work product this is
  claims/claims.json            required — the assertions and conclusions
  provenance/provenance.json    required — who/what produced it
  evidence/                     source records and an evidence index
  facts/                        extracted observations
  judgments/                    professional judgment points
  calculations/                 declared computations
  validations/                  recorded validator results
  approvals/                    review events
  renderings/                   memo.md / workpaper.md — derived views
  signatures/                   experimental in v0.1
```

The manifest declares every part with its path, media type, and SHA-256 digest, so tampering with any declared part is detectable from the manifest alone. All objects validate against published JSON Schemas (draft 2020-12). The container is deliberately boring — ZIP, JSON, SHA-256 — because a trust format's first obligation is to be trivially implementable, in any language, for decades.

3.3 Profiles and assertions

VWP Core is domain-neutral. The **finance profile** (`vwp-finance`) adds the assertion vocabulary familiar from financial reporting and auditing — `existence`, `occurrence`, `completeness`, `valuationAllocation`, `rightsAndObligations`, `presentationDisclosure` — so that every claim in a financial work product can declare *which assertions it speaks to*, and packaged evidence can be mapped to the assertions it supports. Extensions narrow further: an ASC 606 extension adds revenue-specific tie-out validators; a digital-asset extension adds wallet ownership and control checks; a controls extension addresses control-test evidence; a forecast extension structures forward-looking estimates, whose claims assert the integrity of a derivation rather than a past or present state (see the second sidebar in Section 5).

4. The verification model

4.1 Layered, deterministic validation

Package validation — the checks that determine whether a package is valid — runs in layers, all deterministic:

1. **Core validators** apply to every package: the manifest exists and conforms to schema; required parts are present; every declared SHA-256 digest matches the actual bytes; workproduct, claims, and provenance conform to their schemas; every evidence reference in every fact and claim resolves; every claim's references to facts, judgments, and calculations resolve; every external-reference retrieval block is well-formed and carries no embedded secret (Section 4.6); archive paths are safe.
2. **Profile validators** run when the package declares a profile. For `vwp-finance`: every claim should carry at least one assertion; every claim should have at least one supporting fact, evidence reference, calculation, or judgment; every unresolved judgment must surface in the verification result.
3. **Extension validators** run when the package declares an extension — for example, that allocated revenue equals the transaction price, or that every included self-custody wallet has proof-of-control evidence.

4.2 Five statuses, and why `reviewRequired` exists

Every check yields one of five statuses:

Status	Meaning	Effect
`pass`	The check succeeded	—
`fail`	A verifiable property is violated	Package is invalid
`warning`	Permitted but notable (e.g., no signature)	Valid unless strict mode
`reviewRequired`	A human judgment is pending	Valid unless strict mode; prominently surfaced
`notRun`	The check did not apply or could not run	Informational

`reviewRequired` is the load-bearing status. A verification framework for professional work must not pretend that machines can settle questions of professional judgment — and must not let those questions hide inside prose. In VWP, a judgment left unresolved is not a schema violation; it is a *surfaced condition* that every verifier reports and that strict-mode consumers can refuse. The machine's job is to guarantee the human cannot miss it.

4.3 What validation means — and does not mean

VWP validation means that a package conforms to specified schemas and validation rules. It does **not** mean the professional conclusion is correct, complete, audited, legally sufficient, or appropriate for a particular use. A package can be perfectly valid and professionally wrong: built on unreliable evidence, resting on a poor judgment, or omitting what should have been considered. Validation moves the *mechanical* portion of review to machines and makes the *judgmental* portion explicit. Professional judgment, applicable standards, and independent review still govern reliance on any VWP package.

4.4 AI builds; deterministic software verifies

VWP is model-agnostic by construction. AI systems are expected to be prolific *builders* of VWP packages — extracting facts, drafting claims, assembling evidence, writing renderings — and the format is designed to make that safe rather than to prevent it. Provenance records which models and tools participated — the exact provider, model identifier, and version, not a family or marketing name — and under which policies. Exactness here is not pedantry: it is what makes *model diversity* possible downstream. A reviewer who challenges model-assisted work with the same model that produced it inherits that model's correlated blind spots; recording precisely which model prepared the work lets auditors deliberately choose a different one to challenge it. But no validator consults a model, requires network access, or executes package contents. A package produced by a frontier model, a legacy script, or a human with a text editor is verified identically. Trust attaches to the verification, not to the generator.

Models have a second role on the *reviewing* side — challenging a package rather than building it. That role is real, valuable, and categorically different from validation, so VWP gives it its own artifact.

4.5 Analysis attestations: model-assisted review, made inspectable

VWP does not assume that all verification is deterministic. Many professional conclusions depend on contract interpretation, policy application, estimates, and judgment — properties no schema check or tie-out can settle. VWP therefore supports **analysis attestations**: structured, signed records of model-assisted review procedures performed over a defined evidence scope.

An analysis attestation may be generated by the preparer, an auditor, a specialist, or another reviewer. It records the exact model and toolchain used (provider, model identifier, and version), the evidence reviewed, the claims challenged, the findings, the limitations, and the required human disposition. Because the package's provenance already discloses exactly which model prepared the work, a challenger can — and generally should — be a different model, so that preparation and challenge do not share failure modes. A concrete example: an auditor directs an LLM challenger at the ASC 606 package of Section 5 — read the MSA excerpt and the order form, and challenge the claim that implementation services are distinct. The

attestation records which evidence the model actually examined (by evidence ID and hash), what it was asked to challenge, what it found — say, a clause suggesting deeper integration than the judgment's basis acknowledges — what it could not assess, and that the finding requires disposition by the engagement team.

Two boundaries keep this layer honest:

- **Attestations never affect package validity.** Core validation remains deterministic; an analysis attestation cannot make a package valid or invalid. It is itself package content — hashed, signed, carrying its own provenance — subject to the same integrity checks as any other part.
- **Findings route to humans, not to conclusions.** An attestation's output is a structured finding with a required human disposition, the same design posture as `reviewRequired`: the machine's analytical work is surfaced so the human cannot miss it, never silently absorbed into the conclusion.

Analysis attestations are not audit opinions and do not replace professional judgment. They provide structured, reviewable, evidence-linked analytical outputs that can support further procedures and reviewer decision-making — and they make model-assisted review *inspectable* in the same way the rest of VWP makes preparation inspectable: scope declared, evidence pinned, model disclosed, limitations stated, disposition owned by a person.

The v0.2 schema described here now exists as `schemas/v0.1/analysis-attestation.schema.json`, published early; see `spec/attestations.md`.

4.6 Evidence that lives elsewhere: authenticated, long-lived references

The strongest evidence is embedded in the package and covered by the integrity envelope: the verifier confirms the exact bytes the conclusion rests on. But much professional evidence cannot travel inside a package — it is too large, access-restricted, or the authoritative copy must remain in a system of record that outlives any single workpaper. VWP references such material by `externalRef`, and for long-lived sources it describes *how the reference is retrieved*, including authentication, without ever becoming a place secrets are stored.

A retrieval descriptor carries the URL, the HTTP method, and an auth block aligned to the relevant IETF standards: **RFC 6750** for Bearer-token usage and **RFC 9728** for OAuth 2.0 protected-resource metadata discovery (the `.well-known/oauth-protected-resource` document from which a client learns the authorization server, token types, and scopes). Crucially, the descriptor names only *where a credential comes from* — a source reference such as `env:BITWAVE_VWP_TOKEN`, `vault:…`, or `oauth:…` — never the credential itself. A literal secret in that field is a verification **failure**, because packages are shareable and must be treated as untrusted input. Credentials are gathered *over the top* at retrieval time: a verifier enumerates the distinct origins the package's references target, obtains one credential per origin, and only then fetches.

This splits verification into two honest tiers. **Offline verification stays deterministic and network-free** — the default. It checks that every retrieval descriptor is well-formed and leaks no secret, but never reaches the network, so the same package bytes always produce the same result and an auditor can verify with no credentials at all. **Online verification is a separate, opt-in tier.** When a reviewer chooses to run it, the verifier fetches each reference with its declared auth and compares the retrieved bytes against a

declared `contentHash`. That content hash is what turns a live, authenticated, long-lived URL into *verifiable* evidence: the reference is confirmed both reachable and unchanged, without the material or the secret ever entering the package. The online result is reported as distinct from the deterministic core result, so what a machine settled offline is never confused with what depended on a particular network and credential at a particular moment.

5. Worked example: an ASC 606 revenue workpaper

The reference repository includes a fully synthetic example package (`examples/asc606-revenue-contract`): a fictional 24-month, 300,000 USD contract between "DemoCo" and "Acme Corp" comprising a hosted SaaS subscription, implementation services, and premium support, billed upfront.

The package contains three evidence files (an order form, an MSA excerpt, a standalone selling price book), eight extracted facts, five claims mapped to the ASC 606 five-step model, one judgment, two calculations, a recorded validation report, an approval event, and a markdown memo rendering.

The structure carries the analysis:

- **Claim (step 4):** "The transaction price is allocated on a relative standalone selling price basis: 192,000 USD to the SaaS subscription, 60,000 USD to implementation services, 48,000 USD to premium support." The claim declares the `valuationAllocation` assertion and links to three SSP facts (each linked to the price book evidence) and to a calculation.
- **Calculation:** `calc_ssp_allocation` declares its inputs (transaction price 300,000; SSPs 240,000 / 75,000 / 60,000), its outputs, and its method. An extension validator recomputes the tie-out.
- **Judgment:** `j_impl_distinct` — are the implementation services distinct from the subscription? — records the question, the proposed conclusion, its basis, the alternative treatment and its consequence, a risk level, and the approver role required. It is deliberately left in `reviewRequired`.

Verifying the packed artifact:

```
PASS core.manifest.schemaValid
PASS core.requiredParts.exist
PASS core.partHashes.match
PASS core.evidenceRefs.resolve
PASS core.claimRefs.resolve
WARN core.signatures.present – no package signature found
PASS finance.claims.haveAssertions
PASS finance.claims.haveSupport
REVIEW finance.requiredJudgments.complete – 1 judgment requires review: j_impl_distinct
PASS asc606.allocation.tieOut – allocated 300000 equals transaction price 300000
PASS asc606.revenueSchedule.total – schedule total 300000 equals allocated consideration 30000

Summary: valid with review items
```

Every mechanical property a reviewer would otherwise inspect by hand — evidence present and untampered, references resolving, allocation tying, schedule footing — is checked exhaustively in milliseconds. What remains for the human is exactly the item the package refuses to let them miss: the distinctness judgment. If a model had drafted this package and hallucinated a price-book figure, the SSP fact's evidence hash and the allocation tie-out give the reviewer concrete threads to pull; if it had invented an evidence reference outright, verification fails.

> ***Sidebar: digital-asset wallet ownership.*** *A second reference example applies the same structure to a treasury entity's wallet population: existence (balances tie to chain and custodian evidence), completeness (every wallet included or excluded with documented rationale), and rights and obligations. The extension validators check that every included wallet has an entity mapping, every included self-custody wallet has proof-of-control evidence, and every exclusion has a rationale. The example also illustrates VWP's insistence on careful claim language: a signed wallet message demonstrates control of a key at a point in time — it does not, by itself, establish legal title, beneficial ownership, or accounting recognition, which require additional evidence and professional judgment. Encoding that distinction in the claim itself, rather than leaving it to a footnote, is precisely what a structured claim graph is for.*

> ***Sidebar: forward-looking estimates.*** *A forecast extension (`vwp-finance-forecast`) applies the structure to work products whose conclusions cannot be verified against reality at production time at all — the staple deliverables of FP&A: a validator-rewards projection, a revenue run-rate, a budget build-up. Only the chain is verifiable, which makes the decomposition more load-bearing, not less. The calculable core of a forecast — a published emission curve, a contracted rate, a run-rate extrapolation — lives in recomputable scenario calculations; every discretionary departure from observed fact (participation, dilution, pricing) must be an explicit judgment, and a scenario declaring no assumption judgments is flagged for review, because zero acknowledged discretion is the signature of a guess laundered into an input cell, not of certainty. Claims carry an `estimate` assertion that asserts the integrity of the derivation, never the outcome. Input freshness is checked against the forecast's own as-of date — never the verifier's clock, so the same bytes verify identically forever; whether an aging forecast may still be relied on is consumer policy, not validation. And forecasts have a property retrospective workpapers lack: they generate their own future evidence. When the horizon lapses, the successor package pins its predecessor by hash, extracts observed actuals, and carries a variance calculation — an evidence-linked chain of forecast, actuals, and revised forecast in which the judgments that missed are individually identifiable. The substrate of Section 2 accumulates not just verification but a track record.*

6. VWP in the audit workflow

VWP is preparer-side infrastructure with auditor-side benefits. The package is created where the work is done — by the entity, its accounting team, or its systems — and consumed by everyone who must rely on it.

- **Evidence requests and PBC.** A request is answered with a package rather than a folder of files. On receipt, the auditor's tooling verifies integrity, completeness against the declared parts, evidence resolution, and tie-outs — before a human reads a page. Deficient packages are rejected mechanically and early, replacing weeks of back-and-forth.
- **Reperformance.** Declared, recomputable calculations let the auditor reperform arithmetic and allocations independently, with their own tools, at full population scale where the check is mechanical.
- **Version comparison.** Packages are diffable: between a draft and a final, or between this year's and last year's, a structural diff shows which claims changed, which evidence was added or removed, which judgment conclusions moved, and which validations changed status.
- **Controls.** A control-test package carries the reconciliation, the reviewer's signoff, the exception log, and claims that the control was performed, the reviewer was authorized, and exceptions were resolved — with the evidence trail attached.
- **AI-assisted preparation, made inspectable.** When clients use AI to prepare analyses — as they increasingly will — provenance discloses it, and the package structure gives the auditor something to verify other than the model's fluency.
- **Model-assisted challenge.** The auditor's own tooling can direct an LLM challenger at a received package — re-reading the pinned evidence, stress-testing claims and judgment bases — and record the procedure as an analysis attestation (Section 4.5): scope, model, findings, limitations, and the engagement team's disposition. The same package accumulates challenges from both sides of the engagement, each signed and inspectable.

What stays human. Everything that matters most. The sufficiency and appropriateness of evidence, the reasonableness of judgments, the evaluation of estimates, materiality, risk assessment, and the opinion itself are the auditor's, entirely and unchanged. VWP does not perform procedures, and a valid package is an input to professional evaluation, never a substitute for it. VWP validation is not an assurance engagement.

Regulatory and standards context. Standard setters and regulators have spent recent years addressing technology in the audit — from the AICPA's Audit Data Standards for structured data extracts, to revisions of evidence standards contemplating technology-derived information, to regulatory staff publications on audit firms' use of generative AI. A consistent theme runs through them: technology may assist, but the firm remains responsible for the evidence it relies on and the procedures it performs. VWP is designed to fit that posture rather than challenge it. It does not automate judgment or transfer responsibility; it makes the *support structure* of client-prepared work transparent and independently checkable, which is the precondition for using any of it responsibly. Nothing about the format presumes or requires any particular standard-setting outcome.

7. Related work

VWP occupies a gap adjacent to several existing standards rather than competing with them. XBRL structures the *outputs* of financial reporting — statements and facts for regulators and analysts — but not the work products that justify them. The AICPA's Audit Data Standards structure *data extracts* handed to

auditors, not conclusions or their support. C2PA binds provenance to *media assets*; in-toto and SLSA attest to *software supply chain* steps — the closest architectural cousins, sharing VWP's core move of signed, independently verifiable attestations about how an artifact was produced, but aimed at build pipelines rather than professional conclusions. VWP borrows the supply-chain lesson — untrusted producers, deterministic verification, portable attestations — and applies it to the artifact those standards all leave unstructured: the evidence-backed professional work product itself.

8. Design choices and limitations

Deliberately boring. v0.1 is ZIP, JSON Schema, and SHA-256. No RDF, no blockchain anchoring, no novel cryptography. A verification format earns adoption by being implementable in an afternoon and stable for decades; sophistication can be layered on later, behind the same package boundary.

Threat model, honestly stated. Hash validation detects tampering with *declared* parts after packaging. It does not authenticate the author (signatures are experimental in v0.1 and carry placeholder semantics), does not detect content that was wrong before it was hashed, does not validate undeclared files, and cannot make unreliable evidence reliable — garbage-in-garbage-out survives any container format. Packages must be treated as untrusted input: conforming verifiers refuse path traversal and never execute package contents. What the format guarantees is narrower and more valuable than "this is true": *this is exactly what was assembled, its internal structure is coherent, its arithmetic ties, and its open judgments are visible.*

Known gaps in v0.1. Signature and key-discovery semantics are unspecified beyond placeholders; validator coverage is a starting set; evidence locators for external systems are not yet standardized (facts today reference evidence at whole-file granularity, not line or cell level); the analysis attestation object is specified conceptually but its schema is not yet in the v0.1 schema set; the approval model records events but not workflows. These are roadmap items, not oversights — see below.

9. Roadmap and participation

The reference repository contains the v0.1 draft specification, JSON schemas, a TypeScript verifier CLI (`pack`, `unpack`, `verify`, `inspect`, `diff`, `render`), profile documentation, and five synthetic example packages. The near-term roadmap: real signature support, an analysis attestation schema, forecast-extension validators, and conformance tests (v0.2); Python and Go SDKs, standardized evidence locators with intra-file anchoring, and a review-event log (v0.3); converging on a stable format, a validator plugin interface, and a public conformance suite (v1.0).

The format will be shaped by the people who would have to live with it. We are explicitly seeking review from audit professionals — is the object model faithful to how work products are actually built and reviewed? are the statuses right? what would you refuse to rely on? — and from the technical teams that support them — implement a verifier, break the format, tell us where the spec underdetermines behavior. Issues and pull requests are open.

Repository: `github.com/verifiable-work-product/vwp` *— specification under* `spec/`*, schemas under* `schemas/v0.1/`*, reference CLI under* `cli/`*, synthetic examples under* `examples/`*. All example data is fictional. Apache-2.0.*